3C

DESARROLLO DE SOFTWARE

Desarrollo de aplicaciones de Software para la solución de problemas

Saber, valorar, elegir

PADRE ARRUPE
Colegio Español

DOCENTE: Daniel Sosa

COLEGIO ESPAÑOL PADRE ARRUPE

Colegio internacional.

Tercer Año de Bachillerato de Software

Desarrollo de aplicaciones de Software para la solución de problemas

Periodo 1.

GUÍA 5 – Uso de APIs en Java

OBJETIVOS DE LA PRÁCTICA

Objetivo: Comprender la estructura y sintaxis de Java, manejar variables y tipos de datos, utilizar operadores y aplicar estructuras de control de flujo.

INDICACIÓN GENERAL: Introducir variables y tipos de datos, explicar operadores y expresiones, y enseñar estructuras de control de flujo

Nota: La guía forma parte integral de la clase, por lo que su finalización se revisa durante la misma.

INTRODUCCIÓN TEÓRICA

1. Introducción

Java es un lenguaje de programación ampliamente utilizado para el desarrollo de aplicaciones empresariales, web y móviles. Una de sus principales fortalezas es el soporte para el uso de APIs (Application Programming Interfaces), que permiten la comunicación entre diferentes aplicaciones y sistemas. Este reporte analiza el uso de APIs en Java, su importancia y algunos ejemplos de implementación.

2. Importancia de las APIs en Java

Las APIs en Java facilitan la integración con otros sistemas, permitiendo la reutilización de código y la optimización del desarrollo de software. Algunas de sus principales ventajas incluyen:

- **Modularidad**: Facilitan la creación de aplicaciones escalables y mantenibles.
- **Interoperabilidad**: Permiten la comunicación entre aplicaciones escritas en diferentes lenguajes de programación.
- **Eficiencia**: Reducen el tiempo de desarrollo al proporcionar funciones listas para usar.

3. Tipos de APIs en Java

Java ofrece diversos tipos de APIs según su propósito:

- **APIs Estándar de Java (Java SE)**: Incluyen bibliotecas para manipular cadenas, colecciones, entrada/salida (I/O), concurrencia, entre otras.
- **APIs Empresariales (Java EE / Jakarta EE)**: Proporcionan funcionalidades para el desarrollo de aplicaciones web, incluyendo Servlets, JSP, JPA, y Web Services.
- **APIs de terceros**: Librerías externas como Apache Commons, Gson, Jackson, entre otras.

- **APIs REST y SOAP**: Para la comunicación entre servicios web.

4. Ejemplo de Uso de APIs en Java

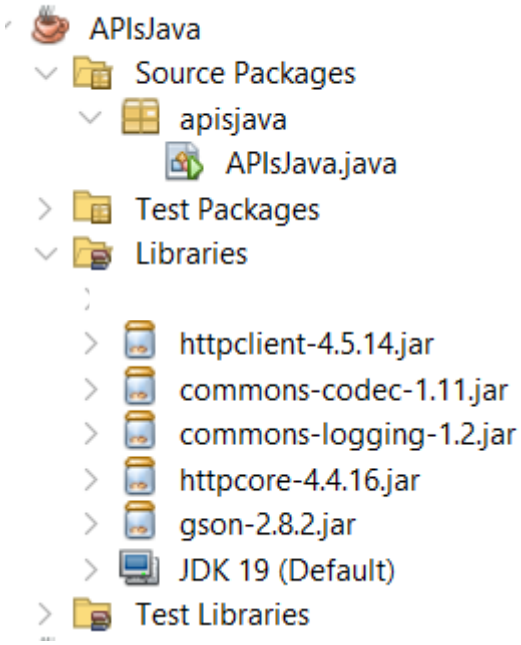
Uno de los usos más comunes de APIs en Java es el consumo de servicios web RESTful. A continuación, se presenta un ejemplo usando `HttpURLConnection`:

```
import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.net.HttpURLConnection;
import java.net.URL;

public class ApiClient {
    public static void main(String[] args) {
        try {
            URL url = new
URL("https://jsonplaceholder.typicode.com/posts/1");
            HttpURLConnection conn = (HttpURLConnection)
url.openConnection();
            conn.setRequestMethod("GET");

            BufferedReader reader = new BufferedReader(new
InputStreamReader(conn.getInputStream()));
            String line;
            while ((line = reader.readLine()) != null) {
                System.out.println(line);
            }
            reader.close();
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Este código realiza una solicitud GET a una API pública y muestra la respuesta en consola.

REALIZACIÓN DE PRÁCTICA

1. Crearemos el proyecto **'APIsJava'** recordando siempre java with ant>java Project.
2. Para este caso, únicamente utilizaremos la clase **Main** para poder consumir la API de Pokemon, si realizamos el proceso de forma correcta recibiremos un archivo JSON con la información general del pokemon seleccionado.
3. Para esta guía utilizaremos varias librerías que serán proporcionadas en el siguiente enlace: https://drive.google.com/drive/folders/1GbaWt5xetOVo4yz8loh_l1u2OCq3hD49?usp=sharing
4. Deberá añadir todas las librerías al proyecto, caso contrario simplemente tendrá un error, la estructura del proyecto deberá quedar de la siguiente forma:



5. Vamos a crear 3 versiones del proyecto, primero consumiremos la API mostrando el JSON obtenido por pokeapi.
6. En nuestra clase Main vamos a agregar el siguiente código:

Version 1:

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Main.java to edit this template
 */
package apisjava;


import org.apache.http.client.methods.CloseableHttpResponse;
import org.apache.http.client.methods.HttpGet;
import org.apache.http.impl.client.CloseableHttpClient;
import org.apache.http.impl.client.HttpClients;
import org.apache.http.util.EntityUtils;
/**
 *
 * @author Daniel Sosa
 */
public class APIsJava {

    /**
     * @param args the command line arguments
     */
    public static void main(String[] args) {
        // TODO code application logic here
        try {
```

```java
            // Crear un cliente HTTP
            CloseableHttpClient httpClient = HttpClients.createDefault();

            // Crear una solicitud GET para obtener información sobre un Pokémon específico (por ejemplo, Pikachu)
            HttpGet request = new HttpGet(uri:"https://pokeapi.co/api/v2/pokemon/pikachu");

            // Ejecutar la solicitud y obtener la respuesta
            CloseableHttpResponse response = httpClient.execute(request);

            // Imprimir el código de estado de la respuesta
            System.out.println("Código de Estado: " + response.getStatusLine().getStatusCode());

            // Leer la respuesta y mostrarla en la consola
            String responseBody = EntityUtils.toString(entity: response.getEntity());
            System.out.println("Respuesta: " + responseBody);

            // Cerrar recursos
            response.close();
            httpClient.close();
        } catch (Exception e) {
            e.printStackTrace();
        }
    }

}
```

7. Al ejecutar nuestra clase obtendremos el siguiente resultado:

```
run:
CÓdigo de Estado: 200
Respuesta: {"abilities":[{"ability":{"name":"static","url":"https://pokeapi.co/api/v2/ability/9/"},"is_hidden":false,"slot":1},{"ability":{"name":"lightning-rod","url":"https://pokeapi.co/api/v2/abili
BUILD SUCCESSFUL (total time: 1 second)
```

Como podemos observar, obtenemos un JSON el cual refleja la información del pókemon.
En el paso #8 vamos a modificar nuestro código para poder mostrar los datos del JSON dentro de un JOption Pane.

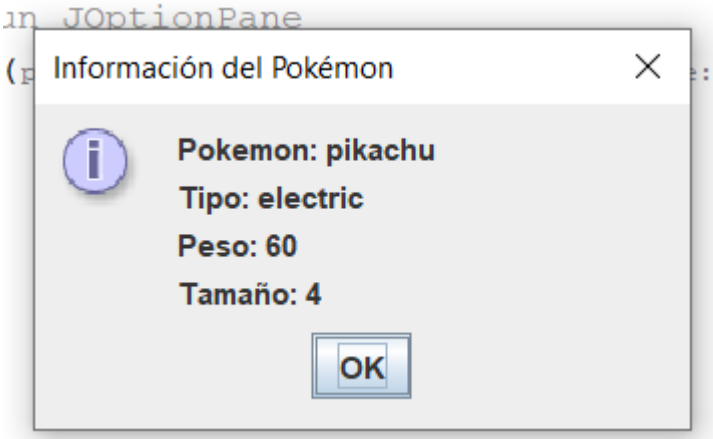
Version 2:

8. En la clase **Main** deberá modificar el código de la siguiente forma:

```java
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Main.java to edit this template
 */
package apisjava;


import javax.swing.JOptionPane;
import com.google.gson.Gson;
import com.google.gson.JsonObject;
import org.apache.http.client.methods.CloseableHttpResponse;
import org.apache.http.client.methods.HttpGet;
import org.apache.http.impl.client.CloseableHttpClient;
import org.apache.http.impl.client.HttpClients;
import org.apache.http.util.EntityUtils;
/**
/**
 *
 * @author Daniel Sosa
 */
public class APIsJava {

    /**
     * @param args the command line arguments
     */
    public static void main(String[] args) {
        // TODO code application logic here
        try {
```

```java
            // Crear un cliente HTTP
            CloseableHttpClient httpClient = HttpClients.createDefault();

            // Crear una solicitud GET para obtener información sobre un Pokémon específico
            //(por ejemplo, Pikachu)
            HttpGet request = new HttpGet(uri:"https://pokeapi.co/api/v2/pokemon/pikachu");

            // Ejecutar la solicitud y obtener la respuesta
            CloseableHttpResponse response = httpClient.execute(request);

            // Leer la respuesta como una cadena JSON
            String responseBody = EntityUtils.toString(entity: response.getEntity());

            // Analizar el JSON utilizando Gson
            Gson gson = new Gson();
            JsonObject pokemonObject = gson.fromJson(json: responseBody, classOfT: JsonObject.class);

            // Extraer la información relevante del objeto JSON
            String pokemonName = pokemonObject.get(memberName:"name").getAsString();
            String pokemonType = pokemonObject.getAsJsonArray(memberName:"types").get(i: 0)
                    .getAsJsonObject().getAsJsonObject(memberName:"type").get(memberName:"name")
                    .getAsString();
            int pokemonWeight = pokemonObject.get(memberName:"weight").getAsInt();
            int pokemonHeight = pokemonObject.get(memberName:"height").getAsInt();

            // Construir el mensaje para el JOptionPane
            String message = "Pokemon: " + pokemonName + "\n" +
                             "Tipo: " + pokemonType + "\n" +
                             "Peso: " + pokemonWeight + "\n" +
                             "Tamaño: " + pokemonHeight;

            // Mostrar la información en un JOptionPane
            JOptionPane.showMessageDialog(parentComponent: null, message, title: "Información del Pokémon",
                    messageType: JOptionPane.INFORMATION_MESSAGE);

            // Cerrar recursos
            response.close();
            httpClient.close();
        } catch (Exception e) {
            e.printStackTrace();
        }
    }

}
```

9. El resultado deberá mostrar el siguiente JOptionPane con la información precisa, solicitada por usted:

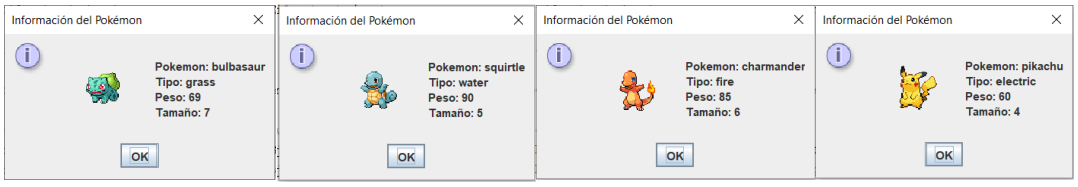


Versión 3:

10. Vamos a modificar nuevamente el código de la clase Main, en la cual ahora le agregaremos el **'sprite'** o imagen del pokemon dentro del jOptionPane:

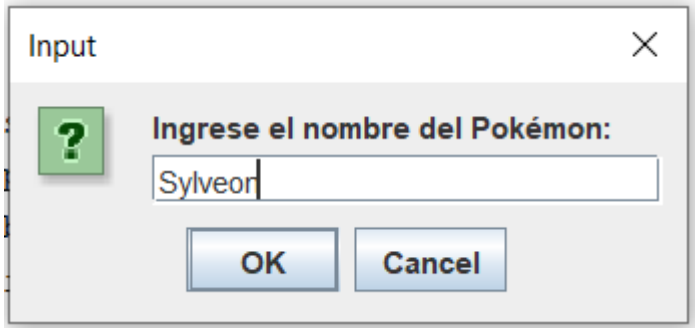
```java
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Main.java to edit this template
 */
package apisjava;

import javax.imageio.ImageIO;
import javax.swing.ImageIcon;
import javax.swing.JLabel;
import javax.swing.JOptionPane;
import com.google.gson.Gson;
import com.google.gson.JsonObject;
import org.apache.http.client.methods.CloseableHttpResponse;
import org.apache.http.client.methods.HttpGet;
import org.apache.http.impl.client.CloseableHttpClient;
import org.apache.http.impl.client.HttpClients;
import org.apache.http.util.EntityUtils;
import java.awt.image.BufferedImage;
import java.net.URL;
/**
 *
 * @author Daniel Sosa
 */
public class APIsJava {

    /**
     * @param args the command line arguments
     */
    public static void main(String[] args) {
        // TODO code application logic here
        try {
            // Crear un cliente HTTP
            CloseableHttpClient httpClient = HttpClients.createDefault();

            // Crear una solicitud GET para obtener información sobre un Pokémon específico (por ejemplo, Pikachu)
            HttpGet request = new HttpGet(uri:"https://pokeapi.co/api/v2/pokemon/pikachu");

            // Ejecutar la solicitud y obtener la respuesta
            CloseableHttpResponse response = httpClient.execute(request);

            // Leer la respuesta como una cadena JSON
            String responseBody = EntityUtils.toString(entity: response.getEntity());

            // Analizar el JSON utilizando Gson
            Gson gson = new Gson();
            JsonObject pokemonObject = gson.fromJson(json: responseBody, classOfT: JsonObject.class);

            // Extraer la información relevante del objeto JSON
            String pokemonName = pokemonObject.get(memberName: "name").getAsString();
            String pokemonType = pokemonObject.getAsJsonArray(memberName: "types").get(i: 0)
                    .getAsJsonObject().getAsJsonObject(memberName: "type").get(memberName: "name").getAsString();
            int pokemonWeight = pokemonObject.get(memberName: "weight").getAsInt();
            int pokemonHeight = pokemonObject.get(memberName: "height").getAsInt();

            // Obtener la URL de la imagen del Pokémon
            String imageUrl = pokemonObject.getAsJsonObject(memberName: "sprites").get(memberName: "front_default").getAsString();

            // Descargar y cargar la imagen desde la URL
            BufferedImage image = ImageIO.read(new URL(spec: imageUrl));
            ImageIcon icon = new ImageIcon(image);

            // Crear un JLabel con la imagen y la información del Pokémon
            JLabel label = new JLabel("<html>Pokemon: " + pokemonName + "<br>Tipo: " + pokemonType + "<br>Peso: " +
                    pokemonWeight + "<br>Tamaño: " + pokemonHeight + "</html>", icon, horizontalAlignment: JLabel.CENTER);

            // Mostrar el JLabel en un JOptionPane
            JOptionPane.showMessageDialog(parentComponent: null, message: label, title: "Información del Pokémon",
                    messageType: JOptionPane.INFORMATION_MESSAGE);

            // Cerrar recursos
            response.close();
            httpClient.close();
        } catch (Exception e) {
            e.printStackTrace();
        }
    }

}
```

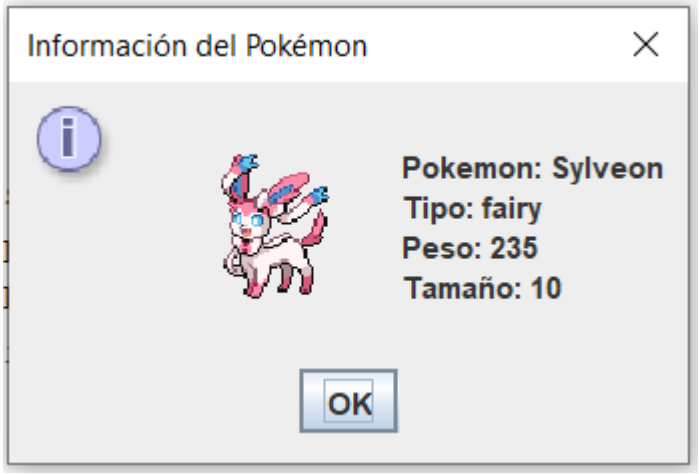
11. El resultado deberá quedar de la siguiente forma, si modifico el nombre del pokemon en la URL este variará su resultado debido al request enviado:

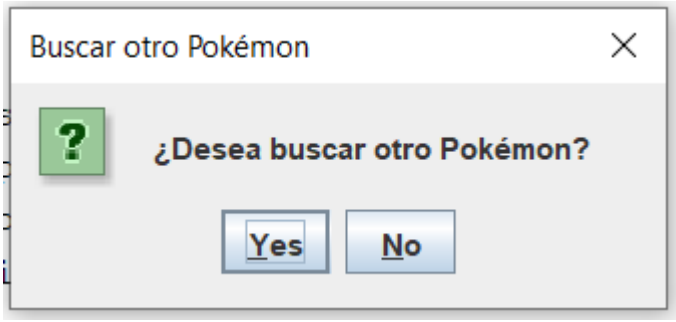


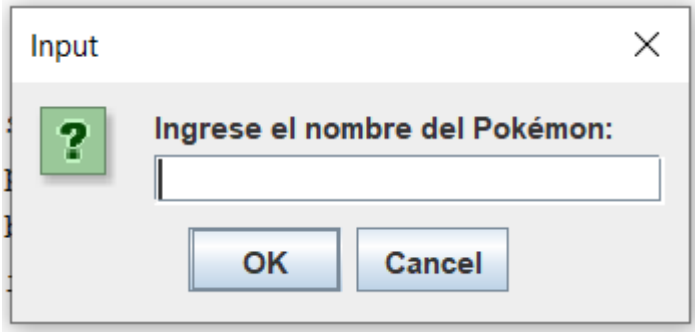
Ejercicio complementario:

Investiga cómo implementar un JOption Pane en el cual el programa solicite al usuario el nombre del pokemon y este le devuelva la información mostrada anteriormente.



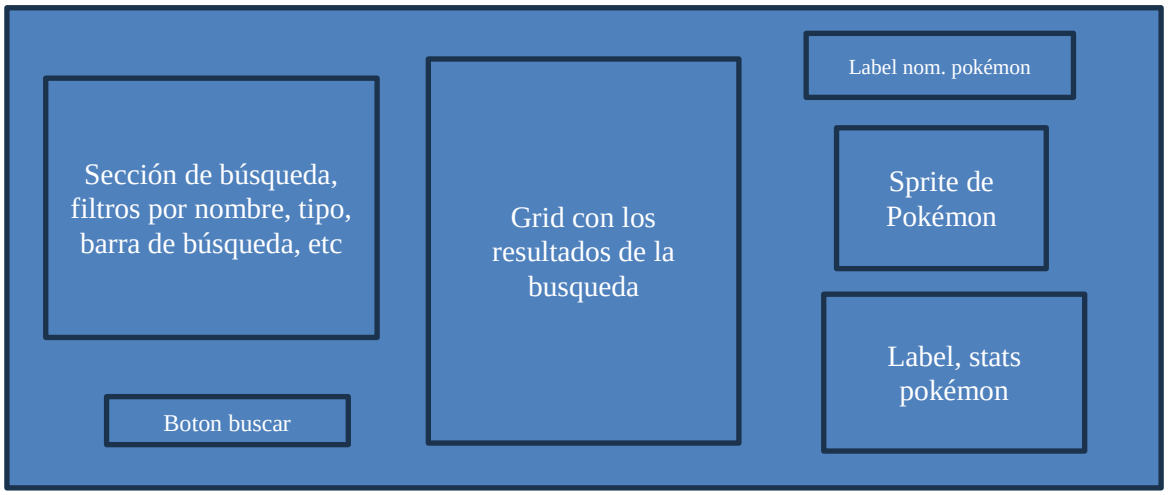






ACTIVIDAD COMPLEMENTARIA

Desarrollar una aplicación utilizando JSwing donde pueda a través de una barra de búsqueda buscar cualquier tipo de Pokémon por nombre o tipo, utilizando la api, el esquema debe ser el siguiente:



BIBLIOGRAFÍA

- Oracle. (s. f.). *Java downloads*. Oracle.com. Recuperado el 4 de marzo de 2024, de https://www.oracle.com/java/technologies/downloads/
- Apache NetBeans. (s. f.). *Apache NetBeans releases*. Apache.org. Recuperado el 4 de marzo de 2024, de https://netbeans.apache.org/front/main/download/index.html
- Rodríguez, & Walter Sagástegui, A. (s. f.). *Configurar Java en Windows. Variables de entorno JAVA_HOME y PATH (CU00610B)*. Aprenderaprogramar.com. Recuperado el 4 de marzo de 2024, de https://www.aprenderaprogramar.com/index.php?option=com_content&view=article&id=389:configurar-java-en-windows-variables-de-entorno-javahome-y-path-cu00610b&catid=68&Itemid=188